Behavioral Issues

Behavior-Driven Development as Engineering Methodology for Agentic Systems
Red-Green-Refactor as Mentorship Loop, Gherkin as Specification Language, and a Methodology That Became a Framework

Larry Klosowski Lauren Mendenhall
Citrate Inc.
Larry@citrate.ai

The Gradient Papers, No. IV
Preprint — not yet peer reviewed

Maturity: [Practiced], strongly. This is a practitioner essay, not a controlled study. What has changed since February 2026 is that the workflow it describes was subsequently codified into the project's standing engineering framework and used to build the rest of the series' software. This revision reports that maturation honestly, and keeps the essay's original candor: the field observations are observations, not statistically significant results.

0.1 Abstract

AI coding agents can generate functionally correct code but lack the engineering discipline that prevents scope drift, architectural violations, and silent regressions over extended development sessions. This paper argues that Behavior-Driven Development (BDD), specifically the Given/When/Then specification language (Gherkin) combined with the red-green-refactor cycle, provides the missing structural discipline for agentic engineering. The argument is grounded in practitioner experience: the author ran three concurrent agentic builds using BDD-first methodology across multiple agent platforms and observed consistent patterns of improvement when agents were constrained by executable specifications rather than free-form instructions. We formalize these observations as a methodology and connect them to the Citrate Network's Mentorship Protocol (Paper III): the test suite is the mentor, the coding agent is the mentee, and passing tests are successful knowledge transfer. Unlike the February 2026 draft, we can now report the sequel: this methodology was codified into the project's **Agentile** engineering framework, a set of non-negotiable rules and workflows, and used to build the software behind Papers I–III and X–XI. This remains a practitioner report; we describe the experimental methodology that would validate the observations, and we have not run it.

Keywords: behavior-driven development, BDD, Gherkin, agentic systems, test-driven development, AI coding agents, red-green-refactor, specification language, software engineering methodology

0.2 1. Introduction

The AI coding-agent shift inverted a classical assumption: that the bottleneck in development is writing code. Agents generate hundreds of lines of plausible code per minute. The new bottleneck

is **specification**, telling the agent what to build precisely enough that the output matches intent, integrates with the existing architecture, and does not silently break what already worked.

This paper's specific claim: Behavior-Driven Development, as formalized by Dan North [1] and implemented through Gherkin [2], is a natural and effective discipline for constraining agentic code generation. The Given/When/Then template forces the developer to specify observable behavior before the agent writes implementation. The red-green-refactor cycle, write a failing test, make it pass, clean up, provides incremental checkpoints that prevent scope drift. And the executable specification (`.feature` files) is living documentation that persists across agent sessions, addressing the context-window problem that plagues long-running builds.

Practitioner basis. The observations come from three concurrent projects run with BDD-first methodology and AI coding agents between October 2025 and February 2026: a HIPAA-compliant healthcare chatbot (IKWE.ai, a separate project by the author, TypeScript/LangChain/Cucumber.js); components of the Citrate Network's consensus layer; and the Polyp Framework, a universal project scaffold for agentic IDEs. These used different agent platforms (Claude Code, Windsurf/Cascade, Cursor, Replit Agent) and different stacks, giving variation, but not controlled experimental conditions.

Implementation status. This paper describes the *builder's* methodology, not a change to the Citrate protocol. We use [**Observation**] for patterns noticed across projects, [**Practice**] for recommended workflows, and [**Hypothesis**] for testable predictions. New in v3: [**Codified**] marks practices that were subsequently made non-negotiable in the Agentile framework (Section 4.3).

0.3 2. Background: BDD and Gherkin

2.1 Origins. BDD was introduced by Dan North (2003–2006) [1] to resolve recurring confusion in Test-Driven Development: where to start, what to test, what not to test. By replacing "test" with "behavior," North reframed the practice from verification to specification, the key insight. BDD draws on TDD's red-green-refactor cycle [3, 4], Evans' Domain-Driven Design and its ubiquitous language [5], and acceptance-test-driven development where acceptance criteria are the definition of done [6].

2.2 The Gherkin language. Gherkin is a structured natural-language format for behavior [2]. Its Given/When/Then template decomposes behavior into preconditions, actions, and expected outcomes:

```
Feature: User Authentication
  As a healthcare provider
  I want to authenticate with MFA
  So that patient data remains HIPAA-compliant

  Scenario: Successful MFA login
    Given a registered provider with email "dr@clinic.org"
    And MFA is enabled for their account
    When they submit valid credentials
    And they enter the correct MFA code
    Then they should receive an authenticated session
    And the session should expire after 30 minutes of inactivity
```

Each scenario is executable, tools like Cucumber.js [7] or pytest-bdd [8] map the steps to functions, so the feature file is simultaneously specification, test, and documentation. That triple

function is what makes Gherkin suited to agentic development, where the agent needs all three in one parseable format.

2.3 The red-green-refactor cycle. Beck's TDD cycle [3]: **Red**, write a failing test (the behavior does not yet exist); **Green**, write the minimum code to pass it; **Refactor**, restructure without changing behavior (the still-passing test confirms it). The property that matters for agentic work is that each phase is a **verifiable checkpoint**: an agent can be stopped and resumed at any phase without losing progress, because each phase's output is committed and independently checkable.

0.4 3. The Agent Discipline Problem

3.1 Observed failure modes. [**Observation**] Across three projects on multiple platforms, five recurring failure modes appeared when agents ran without BDD constraints. **Scope drift:** given "implement authentication," agents expanded scope unasked, adding reset flows, admin panels, each addition reasonable, collectively a much larger system than specified. **Architectural amnesia:** in sessions over ~2 hours, agents forgot earlier decisions, a module specified for dependency injection would be reimplemented with hardcoded dependencies fifty messages later, as earlier decisions fell out of the attention window. **Silent regressions:** modifying existing code to add features broke working functionality without detection or report. **Stub proliferation:** agents created placeholder stubs and lost track of them; a post-build audit of one project found 23 forgotten stub functions. **Platform inconsistency:** the same prose spec produced architecturally incompatible implementations across Claude Code, Windsurf, and Cursor.

3.2 Why BDD addresses these. Each failure maps to a BDD mechanism.

Failure mode	BDD mitigation	Mechanism
Scope drift	Feature file defines exact scope	Agent can only implement specified scenarios
Architectural amnesia	`.feature` files persist across sessions	Specification survives context-window limits
Silent regressions	Prior scenarios stay executable	New code must pass all existing tests
Stub proliferation	Red phase catches unimplemented stubs	Failing tests explicitly surface gaps
Platform inconsistency	Gherkin is platform-agnostic	Same `.feature` file works across all agents

The mitigation is **structural, not behavioral**: it does not require the agent to be "more careful." The `.feature` file is an external artifact that constrains behavior regardless of the agent's internal state. The discipline comes from the process, not the practitioner.

0.5 4. The BDD-First Agentic Workflow

4.1 The workflow. [**Practice**] Across the three projects we converged on: **Step 1 Specification**, the human writes Gherkin feature files (the primary creative act, deciding *what* the system does; the agent does not write these). **Step 2 Red**, the agent writes step definitions and runs them; all fail, confirming the harness works and the behavior is absent. **Step 3 Green**, the agent writes the minimum code to pass, constrained not to anticipate future scenarios. **Step 4 Refactor**, the agent

cleans up while keeping tests green; the developer reviews. **Step 5 Commit and advance**, the passing scenario is committed with a conventional message (e.g. “ FEAT: authentication — MFA login passes”), the next scenario is activated, return to Step 2.

4.2 The developer’s role. The developer shifts from code author to **specification author and quality reviewer**: writing the Gherkin, reviewing the implementation for unintended side effects, and making architectural decisions when the spec admits several valid implementations. This is not a claim about all contexts; for exploratory prototyping or one-off scripts, BDD adds overhead that may not be justified. The claim is specific: for production systems that must stay correct over long agentic sessions, BDD-first reduces the Section 3 failure modes.

4.3 From workflow to framework (new in v3). **[Codified]** The most honest update this revision can offer is that the workflow above stopped being a personal habit and became the project’s standing engineering framework, **Agentile**. Its non-negotiable rules operationalize this paper’s argument: test-first (a failing test precedes implementation), **no mocks or stubs** in delivered code (a direct answer to stub proliferation, with a per-sprint mock budget of zero), **close-with-proof** (a unit of work cannot close without the evidence that its tests pass), and timestamped, authored documents so decisions survive context loss. Two additions go beyond the February draft: **mutation testing**, which checks that the tests themselves actually bite by introducing deliberate faults and confirming a test fails, closing the loophole where a green suite proves little; and **red-green as the atomic unit of work**, so every commit is a verifiable checkpoint. The software behind Papers I–III and X–XI was built under this framework. That is not proof the methodology is optimal, it is evidence that it is livable at the scale of a real system, which is a different and more modest claim.

It is worth noting which rule answers which failure mode from Section 3, because the framework was not assembled abstractly, it accreted one rule per scar. The no-mocks rule and its zero budget are the direct answer to stub proliferation: a delivered stub is a rule violation the build refuses, so the 23 forgotten stubs of the earlier audit cannot accumulate silently. Close-with-proof answers silent regressions: a unit of work does not close on an agent’s assertion that it works, it closes on the evidence that the prior scenarios still pass, so a regression surfaces as a failed gate rather than as a bug discovered weeks later. Timestamped, authored documents answer architectural amnesia across sessions, the decision record outlives the attention window. And mutation testing answers the subtlest failure of all, the green suite that proves nothing: by deliberately corrupting the code and requiring a test to fail in response, it verifies that the tests would actually catch a regression, closing the gap between “the tests pass” and “the tests would notice if they shouldn’t.” None of these rules asks the agent to be more careful; each makes a class of carelessness mechanically impossible to commit.

0.6 5. BDD as Mentorship: connection to Paper III

The BDD cycle maps to the Mentorship Protocol (Paper III). The test suite is the **mentor**, encoding the specification of correct behavior; the coding agent is the **mentee**, receiving guidance (failing tests) and demonstrating learning (passing tests); red-green-refactor is the **mentorship loop**, the mentor identifies a gap (red), the mentee fills it (green), the mentee refines (refactor).

BDD concept	Mentorship Protocol (Paper III)	Citrate Network (Papers I–II)
Feature file	Performance profile	Per-node capability metrics at checkpoint

BDD concept	Mentorship Protocol (Paper III)	Citrate Network (Papers I–II)
Failing test (Red)	Identified weakness	Belnap F/N state on an input class
Passing test (Green)	Successful knowledge transfer	LoRA adapter improves node accuracy
Refactor	Double-loop learning	Routing model restructures coordination
Test suite	Central Oracle's knowledge base	Committed embeddings + state vectors
Commit	Checkpoint	BFT finality checkpoint (interval 50 blocks)

Analogy boundary. This mapping is structural, not operational: a `.feature` file is not literally a performance profile, and a failing test is not literally a Belnap classification. The claim is that both systems implement the same organizational pattern, targeted guidance based on observed weakness, through different mechanisms.

0.7 6. Gherkin for Adapter Validation

[**Practice**] In the Citrate Network, LoRA adapters are generated at checkpoints and distributed to nodes (Paper II §4.3). Before an adapter is registered on-chain, it should pass a validation suite, and we propose expressing that suite in Gherkin, so acceptance criteria are readable by developers, auditors, and governance participants who are not ML specialists:

```
Feature: LoRA Adapter Validation
  Scenario: Adapter improves target domain
    Given a node with baseline accuracy of <baseline> on <domain>
    When the adapter generated at checkpoint <cp> is applied
    Then accuracy on <domain> should be >= <baseline> + <threshold>

  Scenario: Adapter does not degrade other domains
    When the adapter is applied
    Then accuracy on non-target domains should not drop > 5%

  Scenario: Adapter passes fraud-proof verification
    Given the adapter hash committed at checkpoint <cp>
    When any validator replays generation from committed state
    Then the regenerated adapter should match the committed hash
```

This is a proposed application, not an implemented feature; the step definitions require the testing infrastructure of Paper II §8. The registration and commitment path the third scenario checks does now exist on-chain in `LoRAFactory.sol` (`adapterModelCommitment` / `verifyAdapterAt`, Paper III), so the "regenerated adapter matches the committed hash" criterion has a concrete target to test against.

0.8 7. Practitioner Observations

Reported as practitioner observations, not experimental results, and not claimed to generalize beyond the specific projects, platforms, and developer described.

7.1 Agent compliance. [**Observation**] With `.feature` files in the project root, every tested platform recognized Gherkin without explicit instruction and generated matching step definitions. Compliance was highest when the feature file was in the initial prompt context and lowest when it existed only on disk. BDD discipline works best when the specification is in the agent's attention window, not merely available in the filesystem.

7.2 Regression detection. [**Observation**] In the IKWE.ai project, switching to BDD-first mid-project produced a noticeable drop in regressions: before adoption (first ~3 weeks) the author manually caught roughly 4 unflagged regressions per session; after (remaining ~5 weeks) the suite caught most automatically, with roughly 1 per session caught manually. These are approximate counts from memory and logs, not controlled measurements, and we do not know what fraction went undetected in either period.

7.3 Context-window survival. [**Observation**] Feature files preserved architectural decisions across sessions: starting a fresh context with the `.feature` files and existing suite was enough to bring an agent up to speed, where prose onboarding was error-prone and incomplete. The Polyp Framework was built to exploit this, generating feature files as part of scaffolding so every session begins with executable specifications.

7.4 Cross-platform portability. [**Observation**] The same feature files produced compatible step definitions across Claude Code, Windsurf, and Cursor; implementation *style* differed (class-based vs functional), but the behavioral specifications, and therefore the acceptance criteria, were identical.

0.9 8. Experimental Hypotheses

The observations suggest hypotheses to test under controlled conditions; we have not run these experiments. **H1 — BDD reduces agent-introduced regressions:** assign 20 comparable feature tasks to BDD-first vs free-form on the same platform, measure regressions caught by a hidden suite, completeness, and time, with confidence intervals. **H2 — feature files improve cross-session continuity:** build over three sessions, then start session four with feature-files+tests vs a prose summary, measure architectural inconsistencies and time to first productive commit under blind review. **H3 — Gherkin portability:** give five platforms the same ten feature files vs equivalent prose, measure common-suite pass rate, API swappability, and architectural similarity by dependency graph.

0.10 9. Relationship to the Gradient Papers Series

Paper I is the system BDD was used to build. Paper II proposes adapter validation at checkpoints, for which Section 6 offers Gherkin as the specification language. Paper III provides the organizational- learning theory; this paper provides the engineering practice, the test suite is the operational realization of the Central Oracle's mentorship function. Paper IX describes the Strobilation Pipeline, the jellyfish reproductive process of staged differentiation with quality gates at each transition; red-green-refactor is its engineering echo, each cycle producing a more capable system through incremental differentiation, an inspirational analogy, not a formal equivalence.

0.11 10. Conclusion

Behavior-Driven Development is a natural, effective discipline for AI coding agents. Gherkin gives developers a structured way to express intent that survives context-window limits, works across platforms, and produces executable acceptance criteria; red-green-refactor provides incremental checkpoints that prevent scope drift, detect regressions, and surface stubs; and the BDD-as-mentorship

mapping ties this practice to the organizational-learning theory of Paper III. The new fact this revision can add is that the methodology was livable enough to become the framework the rest of the series' software was built under, evidence of practicality, not a controlled proof of superiority. The observations here are from a single practitioner across three projects; the Section 8 experiments describe how to test them rigorously. Until then, this is a practitioner report, useful as guidance, honest about its evidential limits.

0.12 Acknowledgments

Drafting and literature triage were assisted by AI systems; all observations are the authors' own from real projects, and all mechanical claims about the Citrate contracts were verified against the referenced source on chain 40204. This work received no external funding.

0.13 References

[1] North, D. (2006). Introducing BDD. *Better Software Magazine* / dannorth.net. [2] Wynne, M., & Helmé, A. (2012). *The Cucumber Book: Behaviour-Driven Development for Testers and Developers.* Pragmatic Bookshelf. [3] Beck, K. (2002). *Test-Driven Development: By Example.* Addison-Wesley. [4] Astels, D. (2003). *Test-Driven Development: A Practical Guide.* Prentice Hall. [5] Evans, E. (2003). *Domain-Driven Design.* Addison-Wesley. [6] Adzic, G. (2009). *Bridging the Communication Gap: Specification by Example and Agile Acceptance Testing.* Neuri. [7] Cucumber.js (2025). Cucumber for JavaScript. github.com/cucumber/cucumber-js [8] pytest-bdd (2025). BDD library for pytest. github.com/pytest-dev/pytest-bdd [9] Pereira, L., et al. (2018). Behavior-Driven Development benefits and challenges: reports from an industrial study. *EASE.* [10] Irshad, M., Britto, R., & Petersen, K. (2021). Adapting BDD for large-scale software systems. *Journal of Systems and Software*, 177, 110944. [11] Binamungu, L. P., Embury, S. M., & Konstantinou, N. (2018). Maintaining BDD specifications: challenges and opportunities. *SANER.* [12] Klosowski, L., Mendenhall, L. (2026). Citrate: Protocol Specification. *The Gradient Papers No. I* (this series). [13] Klosowski, L., Mendenhall, L. (2026). Paraconsistent Consensus. *The Gradient Papers No. II* (this series). [14] Klosowski, L., Mendenhall, L. (2026). The Mentorship Protocol. *The Gradient Papers No. III* (this series). [15] Klosowski, L., Mendenhall, L. (2026). The Medusa Paradigm. *The Gradient Papers No. IX* (this series). [16] Senge, P. M. (1990). *The Fifth Discipline.* Doubleday. [17] Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022.* [18] Couto, T., et al. (2022). On the characterization of BDD adoption benefits: a multiple case study. *SBQS.*

0.14 Appendix A: Cross-Paper Parameter Consistency (reconciled against code, Aug 2026)

Parameter	Value	Source
Block time	~2 s (testnet)	Paper I; `testnet-config.toml:4`
Checkpoint interval	50 blocks	Paper I; `checkpoint.rs:101`
BFT committee / quorum	100 / 67	Paper I; `checkpoint.rs:88`
Adapter registry + commitment	`LoRAFactory.sol` `0x6e564d22…`	Paper III (corrected from "0x1003 precompile")

Parameter	Value	Source
Adapter rank (r)	16 (default)	Paper II
Adapter regression threshold	5% (proposed)	This paper §6

This paper is part of the Gradient Papers series, published by Citrate Inc. Correspondence: Larry@citrate.ai